

OpenResty L7 Layer: 범용 Gateway 밖에 둔 조직 정책 경계

L7은 proxy가 아니라 tenant, stage, routing, canary, 장애 대응 정책이 만나는 운영 경계다.

요약

Kong, Nginx, API Gateway 계열 도구는 성숙한 선택지다. 그러나 이 프로젝트에서 필요한 것은 범용 gateway 기능이 아니라 조직의 tenant, stage, app, route, canary, blue-green, emergency block 정책을 적용하는 L7 운영 계층이었다. OpenResty를 선택한 이유는 Lua 기반으로 요청 경로에서 정책을 가볍게 적용하고, DUCOS·배포·장애 모니터링과 직접 연결하기 위해서였다.

핵심 판단은 “직접 gateway 제품을 만들겠다”가 아니었다. 범용 gateway 앞뒤에 놓인 조직 정책을 어디에서 통제할 것인가의 문제였다.

문제 정의

업무 플랫폼은 단순 URL routing만으로 운영되지 않는다. 요청에는 고객사, app, stage, 사용자 권한, 배포 버전, canary 대상, 차단 정책, 장애 상태가 함께 걸린다. 이 정책을 application 내부에 넣으면 모든 서비스가 동일한 운영 정책을 반복 구현해야 한다. 반대로 범용 gateway 설정만으로 처리하면 조직의 업무 맥락을 표현하기 어렵다.

필요한 계층은 다음 책임을 가져야 했다.

- tenant, app, stage 기준으로 요청을 분리한다.
- blue-green, canary, emergency rollback과 연결된다.
- 장애 상황에서 특정 고객사나 route를 빠르게 제한한다.
- DUCOS 설정 변경과 runtime 정책을 반영한다.
- application code를 수정하지 않고 운영 정책을 적용한다.

왜 Kong이나 일반 Nginx만으로 끝내지 않았는가

Kong은 plugin, service, route, consumer 모델이 정교하다. 일반 Nginx도 reverse proxy로 매우 안정적이다. 그러나 이 프로젝트에서 필요한 것은 gateway 제품의 기능 목록이 아니라 조직 정책의 실행 위치였다.

Kong을 쓰더라도 tenant-stage 정책, 배포 bundle version, app별 route 상태, emergency block, 내부 audit 기준을 별도 계층으로 설계해야 했다. 그 책임을 Kong plugin에 모두 넣으면 plugin이 업무 정책 서버가 되고, application에 넣으면 운영 정책이 서비스마다 흩어진다.

OpenResty는 이 사이의 경계 계층으로 적합했다. 요청을 가볍게 검사하고, 선언형 정책을 읽고, upstream을 결정하고, 필요한 audit 정보를 남기는 역할에 집중시킬 수 있었다.

책임 경계

계층	책임
OpenResty L7	tenant/stage/app/route 판단, canary 전환, 차단 정책, upstream 결정
DUCOS	L7 정책 설정과 runtime refresh 기준 제공
Deployment	blue-green, canary, versioned bundle 상태 제공
Application	업무 로직, transaction, 도메인 validation 수행
Monitoring	L7 판단 결과와 backend 장애 이벤트를 연결

핵심 설계 결정

정책은 Lua 코드가 아니라 선언형 데이터로 분리했다

OpenResty를 사용한다고 해서 모든 정책을 Lua 코드에 넣으면 운영 변경이 다시 코드 배포가 된다. 따라서 tenant, stage, route, canary, block, upstream 정책은 선언형 데이터로 분리하고 Lua는 실행 엔진으로 제한했다.

```
request
-> tenant/app/stage resolve
-> route policy lookup
-> canary / blue-green decision
-> block / rate limit decision
-> upstream selection
-> audit event
```

이 흐름은 application보다 앞에서 실행되어야 하며, 실패 시 기본 정책과 fallback이 명확해야 한다.

L7은 장애 대응의 첫 번째 차단면이다

장애 상황에서 application code를 수정해 배포하는 방식은 늦다. 특정 route, 고객사, stage, 배포 버전에 문제가 있으면 L7에서 traffic을 제한하거나 이전 upstream으로 전환할 수 있어야 한다. 이 기능은 성능 최적화가 아니라 운영 안전장치다.

배포 시스템과 분리하지 않았다

Blue-green과 canary는 배포 시스템만의 기능이 아니다. 실제 traffic을 어느 backend로 보낼지는 L7이 결정한다. 따라서 versioned bundle, 승인 상태, canary 대상, health check 결과를 L7 정책과 연결했다.

버린 선택과 이유

모든 문제를 범용 API Gateway로 해결하는 방식은 채택하지 않았다. Kong이나 Nginx 기반 gateway는 routing, 인증, rate limit에는 강하지만, 조직별 tenant, stage, canary, 긴급 차단, 운영 승인 이력을 제품 정책과 함께 표현하려면 별도 정책 모델이 필요했다.

application 내부에서 routing 정책을 처리하는 방식도 피했다. 장애가 발생했을 때 application이 살아 있어야 차단이나 우회가 가능하다면 L7 경계의 의미가 약해진다. 트래픽 차단, canary 중단, 특정 tenant 우회는 application 앞단에서 처리되어야 했다.

Ingress 설정만으로 운영 정책을 관리하는 방식도 충분하지 않았다. YAML 변경은 인프라 관점에서는 명확하지만, 운영자가 고객사·업무·배포 버전 기준으로 변경 사유와 영향을 확인하기 어렵다.

L7에서 남겨야 하는 기록

L7 계층은 request를 전달하는 통로가 아니라 변경과 장애를 연결하는 관측 지점이다. 최소한 다음 기록이 남아야 했다.

기록	목적
route policy version	어느 정책으로 traffic이 흘렀는지 재현한다.
tenant/stage 매핑	장애가 전체 문제인지 특정 고객사 문제인지 구분한다.
canary 비율과 변경자	traffic 전환의 책임과 시점을 audit한다.
block/allow 사유	긴급 차단이 임시 조치인지 정책 변경인지 구분한다.
upstream health와 latency	application 장애와 L7 routing 문제를 분리한다.

운영 검증

L7 계층에서 가장 조심한 부분은 책임 과잉이었다. L7이 업무 로직을 이해하기 시작하면 application server의 책임을 침범한다. 따라서 L7은 route, tenant, stage, 배포 상태, 차단 정책까지만 다루고, 업무 validation과 transaction은 application에 남겼다.

두 번째 검증은 실패 기본값이었다. 정책 저장소를 읽지 못할 때 전체 traffic을 열어둘지, 닫을지, 마지막 정상 정책을 유지할지 결정해야 한다. 이 기준은 route 성격과 stage에 따라 달라야 했다.

세 번째 검증은 관측성이다. 요청이 어느 정책에 의해 어느 upstream으로 갔는지 남지 않으면 canary 장애를 설명할 수 없다. L7 판단 결과를 audit event로 남기고 장애 모니터링과 연결했다.

역할과 책임

OpenResty 기반 L7 계층의 문제 정의, tenant/stage/app/route 정책 모델, DUCOS 연동, canary·blue-green 전환 기준, emergency block, L7 audit event 구조를 설계했다.

설계 결론

L7 계층은 proxy 설정 파일이 아니다. 운영 중 traffic을 통제하는 정책 실행면이다. 범용 gateway를 쓰는 것과 조직 정책을 통제하는 것은 별개의 문제이며, 이 프로젝트에서는 그 경계를 OpenResty 기반 L7로 분리하는 것이 적합했다.